Mandate: Research Dossier and Build Blueprint

Spending authority for AI agents on PayPal. Prepared October 21, 2026 for the PayPal AI Hackathon on Devpost. Items marked (verified) come from PayPal, Devpost or vendor pages read during this research. Items marked (unverified) need a test or a check before you rely on them.

1. Executive summary

Verdict. Mandate is a strong entry if you hold the scope. The problem is documented, the PayPal pieces you need exist, and the demo story (a clean purchase, a live block, a refund, then numbers) fits in under three minutes. The main danger is building too much, not building the wrong thing.

1. **The problem is real and recent.** Guardio Labs showed an agentic browser completing a purchase on a fake store with saved card details and no confirmation, and following hidden instructions planted in a page. Palo Alto Networks' Unit 42 reported real-world indirect prompt injection cases against agents in 2026 (twelve detected). OpenAI's security chief has publicly called prompt injection an unsolved problem. Limits enforced outside the model are a sound answer to that.
2. **The protocol layer is crowded; the runnable-enforcement layer is not.** Google AP2, OpenAI/Stripe ACP, Visa Intelligent Commerce, Mastercard Agent Pay and PayPal's own Agent Ready all define how agents authorize or pay. None of them is a small engine a judge can attack in a browser, paired with a recovery flow for mistakes that stay inside the limits. That is your gap.
3. **PayPal feasibility holds on paper (verified).** Vault lets a buyer approve once, after which the merchant side can charge the saved PayPal account without the buyer present. Partial refunds are one API call. Webhooks are signed. The Agent Toolkit ships create_order, capture, create_refund and list_transactions for Python and TypeScript.
4. **Three fixes before you build.** (a) The name collides with AP2's "mandates", and PayPal judges know AP2. (b) Render's free Postgres expires after 30 days and free web services sleep, but the rules require the project to stay available until judging ends on Dec 15. (c) Your plan's Oct 13 gate test date has passed, so run it now.

5. **Four additions that cost little and score well.** Bind each capture to a hash of the approved cart. Reserve budget inside a database transaction. Add an "ambiguity gate" as policy check 9. Base recovery refunds on avoided return cost instead of fixed percentages.

If you do one thing this week: prove the vault flow end to end in sandbox: buyer approves once, server charges with the saved token, partial refund succeeds, signed webhook arrives and verifies. Everything else depends on it.

2. Hackathon facts that shape the build

Item	Detail
Deadline	Nov 12, 2026, 12:00 PM PST, which is Nov 13, 1:30 AM IST. Submit by Nov 10 to keep a two-day buffer.
Submission opened	Oct 1, 2026
Judging period	Dec 1 (2:00 PM PT) to Dec 15 (8:00 AM PT), 2026, per the official rules. Winners around Dec 21. Some listing sites show different judging dates; trust the Devpost rules.
Eligibility	Age of majority in your country. India is eligible. Teams allowed; a team appoints one Representative to submit.
Must use	PayPal developer platform (free sandbox) and an AI tool, both meaningfully.
Functional demo	A hosted URL, or complete run instructions in the repo. A mockup, static prototype or non-functional demo does not count.
Repo	Public GitHub, GitLab or Bitbucket, with all source, assets and instructions, and a license file visible in the About section.
Video	Under 3 minutes, public on YouTube, shows the project working on its device, no third-party trademarks or copyrighted music. The rules say "should be less than" three minutes and judges need not watch beyond it. Aim for 2:50 or less.

Item	Detail
Text	Description of features and functionality, plus which AI tools were used and how. English, or with English translation.
Availability	The project must be free to use and unrestricted for the Sponsor, Administrator and judges until the judging period ends.
Judges may not run it	The rules say judges can score from the text, images and video alone. The video and description carry real weight.
After the deadline	The submission cannot be altered, though you can keep updating your Devpost portfolio.
Existing projects	Must be significantly updated after Oct 1, with the updates explained. Mandate is new, so this should not apply.

Judging criteria (weights not published): technological implementation, design, potential impact, innovation/idea, presentation.

Prizes Mandate can target. Best Use of Agentic Commerce $5,000; Best Use of PayPal + AI $5,000; Best Demo Delivery $5,000; Most Creative $5,000; Most Impactful $5,000; 1st $12,000, 2nd $8,000, 3rd $5,000. Sponsor awards: AG Grid $5,000 / $2,000 / $1,000 (three winners at third place), Channel3 $1,500, Render credits $1,000 / $750 / $500, APIMatic $1,000 (three winners) plus a six-month subscription. Whether one project can win both an overall and a category prize is not stated on the page (unverified); read the rules.

3. The problem, with evidence

An agent that can pay is a different security object from an agent that can read. Today it usually gets a card or a broad token. Three kinds of failure follow.

- **Hijack.** Hidden instructions in a page or email redirect the agent. Guardio's "PromptFix" test hid instructions inside a fake CAPTCHA. Brave reported similar indirect injection against Comet and other agentic browsers.
- **Gullibility.** Guardio told Comet to buy an Apple Watch and it completed checkout on a fake Walmart-style store, using saved card and address, without asking. Safe Browsing did not stop it.

- **Honest error.** The agent buys the wrong thing inside every limit: a 120 cm desk instead of 160 cm, an ISO keyboard instead of ANSI. No injection is involved, so a detector would not fire. The buyer blames the agent, the seller points to a valid order, and the only tool left is a chargeback.

The design response is two layers. Hard limits enforced by plain code bound the loss. A recovery flow handles the mistakes that stay inside the bounds. Your proposal's central claim, containment rather than injection detection, is the right one: the engine judges the action (merchant, amount, category, price), not the agent's reasoning.

4. Landscape and positioning

Player	What it is	Relation to Mandate
Google AP2 (announced Sept 16, 2025; 60+ partners including PayPal, Mastercard, American Express, Adyen)	Open protocol. Three signed mandates (Intent, Cart, Payment) carried as verifiable credentials, giving a non-repudiable trail from intent to payment. Supports human-present and human-not-present purchases.	Closest in vocabulary. AP2 defines the signed record. It does not ship a runnable policy engine or a recovery flow.
OpenAI + Stripe Agentic Commerce Protocol	How an agent and merchant complete a purchase inside chat (Instant Checkout).	Checkout protocol, not spending control.
Visa Intelligent Commerce, Mastercard Agent Pay (April 2025)	Network credentials and tokens for agent purchases, with controls such as per-transaction limits and transaction counts at authorization.	Card rails. Mandate runs on PayPal.
PayPal Agent Ready and Store Sync (announced Oct 28, 2025)	Merchant side. Existing PayPal merchants accept agent-originated payments and become discoverable on Perplexity, ChatGPT, Copilot and others.	Complementary. Mandate is buyer side.

Player	What it is	Relation to Mandate
Agent virtual cards (Privacy, Ramp, Stripe Issuing)	Card-level limits enforced at authorization.	Nearest to "limits", but they check amount, merchant and time, not catalog truth, and they do not recover mistakes.
Coinbase agentic wallets, x402	Crypto rails with session caps.	Different rail.

Positioning line. AP2 and the card networks say how an agent proves authority. Mandate is the part that enforces it and fixes what it cannot prevent, on PayPal, in a form a judge can attack.

Why not an OAuth scope or a card limit? A scope says which API the agent may call. A card limit says how much it may spend. Neither knows what is being bought. Mandate re-fetches price and category from a catalog source, checks the cart against the grant, and logs why.

Name. Because AP2 already uses "Intent Mandate" and "Cart Mandate", either rename the project or state plainly that it enforces AP2-style mandates. Options: Warrant, Ceiling, Tollgate. Search Devpost and GitHub before committing. Claiming AP2 compatibility is only honest if you implement an import or export; treat that as a stretch goal.

5. Concept and threat model

Safety rule. The agent proposes and the engine decides. Limits come from fixed code. The LLM never has free control of money, in prevention or in recovery.

Six stages. Grant (human sets the mandate), Propose (agent builds a cart), Verify (rules check it), Execute (PayPal order and capture), Audit (every decision logged), Recover (refund in-bounds errors).

Credential custody is the whole game. The agent must never hold PayPal credentials or the vault token. They live only inside the Mandate API. The SDK wrapper `guard()` is a real boundary only if it routes the agent's tool calls to the Mandate API instead of handing it raw PayPal tools. If the agent can reach PayPal directly, the engine can be bypassed. Say this in the README.

Threat	Example	What stops it	Residual risk
Prompt injection to buy something else	Page says add an $800 gift card	Checks 2, 3, 4, 5	None for amount and merchant bounds
Mislabeled category	Agent calls a gift card "office furniture"	Check 5 (catalog category)	Catalog source wrong
Inflated or stale price	Cart says $120, catalog says $320	Check 6	Price changes between check and capture (see cart hash)
Runaway loop	Agent retries and buys 40 times	Checks 2, 7, 8	Low
Duplicate charge	Retry after timeout	Check 8 and PayPal-Request-Id	Low
Cart swapped after approval	Agent changes line items before capture	Cart hash binding (section 7)	Low
Race on budget	Two parallel proposals each fit alone	Budget reservation in a transaction	Low
Stolen mandate token	Attacker calls the API	Short-lived, per-session tokens, rate limits	Medium for a public demo
Wrong item within limits	Wrong desk width	Recovery module	Needs human accept
Compromised engine or database	Server breach	Out of scope	Disclose

Out of scope (say so openly): escrow, micropayments, merchant reputation scoring, real money, production vaulting, and any claim to detect injection.

6. PayPal integration: what the docs confirm

Authentication. OAuth 2.0 client credentials. Token URL in sandbox: `https://api-m.sandbox.paypal.com/v1/oauth2/token`. Sandbox base URL `https://api-m.sandbox.paypal.com`, live `https://api-m.paypal.com`. Never mix them.

Orders v2. `POST /v2/checkout/orders` with `intent: CAPTURE`, then `POST /v2/checkout/orders/{id}/capture`. Send a unique `PayPal-Request-Id` header on every create, capture and refund call. That is PayPal's idempotency mechanism, and it is what your check 8 should pair with.

Vault (the autonomy enabler, verified). PayPal documents this flow for charging a returning customer without the customer approving each payment: (1) create an order with `payment_source.paypal.attributes.vault` set to `store_in_vault: ON_SUCCESS`, `usage_type: MERCHANT`, `customer_type: CONSUMER`, plus `experience_context` return and cancel URLs; (2) the buyer approves once through the `payer-action` link; (3) capture, then read the token id at `payment_source.paypal.attributes.vault.id`; (4) later, create orders that reference that saved token and capture them server-side. PayPal's own wording: the customer approves once, then you can charge their account without them present. This maps exactly onto your "Grant" screen: the grant step is the one-time sandbox buyer approval.

Two cautions. First, the docs show the later charge in two shapes: `payment_source.paypal.vault_id` in one version and a `token` object with `type: PAYMENT_METHOD_TOKEN` in another. Test which your sandbox accepts and pin that in the README. Second, the live-environment vault docs mention a PayPal vetting step; sandbox app settings must have Vault enabled. Neither blocks a sandbox demo, but write it down. A fallback exists if vaulting fails in your sandbox: a pre-approved sandbox buyer and an honest README note, as your risk table already says.

Refunds (Payments v2, verified). `POST /v2/payments/captures/{capture_id}/refund`. An empty body refunds the full remaining amount. An `amount` object makes a partial refund. Optional `invoice_id` and `note_to_payer`. Success returns 201 with a refund id and status, normally COMPLETED. Multiple partial refunds are allowed up to the captured total. Over-refunding returns 422 (refund amount exceeded), which is a free second safety net behind your own range check. The order must have been created, approved and captured first.

Webhooks (verified). Register an HTTPS endpoint (port 443 only) per environment and keep the webhook ID PayPal assigns. Useful events: `PAYMENT.CAPTURE.COMPLETED`, `PAYMENT.CAPTURE.DENIED`, `PAYMENT.CAPTURE.PENDING`, `PAYMENT.CAPTURE.REFUNDED`, `PAYMENT.CAPTURE.REVERSED`, `CHECKOUT.ORDER.APPROVED`, and vault events such as `VAULT.PAYMENT-TOKEN.CREATED`. Verify each delivery either with `POST /v1/notifications/verify-webhook-signature` (send auth algo, cert URL, transmission id, signature, transmission time, webhook ID and the raw event) or locally by fetching the certificate and checking the RSA-SHA256 signature; if local, validate that the cert host ends in paypal.com. Rules that bite: simulator events fail signature verification, so test with real sandbox transactions; PayPal retries failed deliveries up to 25 times over about three days, so dedupe on event id; events fire only for the REST app that performed the action, so use one app's credentials consistently; the Orders response returns before the vault token is finished saving, which is why the vault webhook exists. Local development needs an HTTPS tunnel.

Agent Toolkit and MCP (verified). `npm install @paypal/agent-toolkit`. Works with OpenAI Agents SDK, Vercel AI SDK, LangChain, CrewAI and MCP, in TypeScript and Python. Tools include create_order, get_order, pay_order (capture), create_refund, get_refund and list_transactions, plus invoicing and others. It reads `PAYPAL_ACCESS_TOKEN` and `PAYPAL_ENVIRONMENT`, so you must refresh the token yourself. A remote MCP endpoint exists at `https://mcp.sandbox.paypal.com/sse` (production `https://mcp.paypal.com/sse`), and the local server accepts a `--tools` argument that limits which tools are enabled. That flag is a natural hook for `guard()`. Use the toolkit inside the Mandate API, not in the agent's hands.

Gate test (run today, about two hours).

1. Create a sandbox app; enable Vault; note client id and secret; create business and personal sandbox accounts.
2. Create an order with vault request; open the `payer-action` link; log in as the sandbox buyer; approve.
3. Capture; save the vault token id.
4. Create a second order using the token; capture with no browser step.
5. Refund 60% of that capture with a fresh `PayPal-Request-Id`.
6. Register a webhook through a tunnel; confirm capture and refund events arrive and verify.

If step 4 fails, note the exact error, try the other token shape, then fall back.

7. Architecture and design decisions

Components. React dashboard with AG Grid ledger; attack simulator; shopping agent (LLM with tool use); Mandate API on FastAPI; policy engine (plain Python); catalog reconciliation (Channel3); checkout executor; webhook handler; recovery service; PostgreSQL (SQLite locally); PayPal sandbox.

Decisions worth adding to your proposal.

- **Cart hash binding.** When a proposal is approved, store a SHA-256 of the canonical cart JSON (line items, totals, currency, merchant). The executor takes only a `proposal_id`, rebuilds the order from the stored snapshot, and refuses if the hash differs. The agent never supplies order data at execution time. This is your answer to "what stops a swap after approval", and it echoes AP2's cart mandate.
- **Budget reservation.** On APPROVE, inside one database transaction (`SELECT ... FOR UPDATE` on the mandate row), add the amount to `reserved`. On CONFIRMED move it to `spent`. On failure or timeout release it. Without this, two simultaneous proposals can each pass check 2 and overspend together.
- **Money handling.** Integer cents or Decimal, never floats. USD only for the demo; reject other currencies. Totals include tax, shipping and fees, because the captured amount will.
- **Hash-chained audit log.** Each row stores `prev_hash` and `row_hash`. A verify button re-computes the chain. It is cheap and makes "every decision logged" checkable. Both modules write to it.
- **Short-lived agent tokens.** Each visitor session gets its own mandate and a token scoped to it. The agent carries that token and nothing else.
- **Truthful states.** Show CAPTURED from the synchronous response, then CONFIRMED after a verified webhook, and reconcile by polling `get_order` if the webhook is late.

State machines. Proposal: RECEIVED, DENIED, ESCALATED, APPROVED, EXPIRED. Payment: RESERVED, ORDER_CREATED, CAPTURED, CONFIRMED, FAILED, RELEASED. Case: OPEN, DIAGNOSED, PROPOSED, ACCEPTED, REJECTED, REFUNDED, ESCALATED_TO_DISPUTE.

API surface (suggested). `POST /mandates`, `GET /mandates/{id}`, `POST /mandates/{id}/revoke`, `POST /proposals`, `POST /proposals/{id}/approve` (human, for ESCALATE), `GET /ledger` (stream via SSE or WebSocket), `POST /cases`, `POST /cases/{id}/accept`, `POST /cases/{id}/reject`, `POST /webhooks/paypal`, `POST /attacks/run`, `POST /reset`.

SDK shape. Keep your three-line snippet. Behind it, `guard(tools, mandate)` replaces create_order and pay_order with calls to `/proposals`, and omits create_refund from the agent's tool list entirely, since refunds belong to the Recovery path.

8. Policy engine specification

Every proposal runs the same ordered checks. All plain code. Results are APPROVE, ESCALATE or DENY, with rule ids stored. An LLM intent check may only move a result toward ESCALATE.

#	Check	Logic	On fail
1	Mandate active	status is ACTIVE and now is before expires_at	DENY
2	Remaining budget	total plus reserved plus spent is at most budget	DENY
3	Per-transaction cap	total is at most per_txn_cap	DENY
4	Merchant allowlist	merchant id is in the list	DENY, or ESCALATE if the mandate allows new merchants with approval
5	Category	catalog category is in allowed categories; ignore the agent's label	DENY
6	Reconcile	each line item exists in the catalog; unit price within tolerance (for example 1% or 50 cents); quantity within a cap; tax, shipping and fees included in total; currency matches	DENY
7	Velocity	purchases in the last hour below the limit	DENY
8	Idempotency	no matching cart hash in a recent window	DENY, return the earlier result

#	Check	Logic	On fail
9	Ambiguity gate (new)	for each constraint in the instruction (for example width 160 cm), the catalog record must contain that field; if missing, ask the human	ESCALATE
10	Recurring billing (new)	item or plan is a subscription or auto-renew and the mandate does not allow it	DENY
11	Near-limit (new)	total above 80% of the cap, or first purchase from a merchant	ESCALATE

Check 9 in detail. An LLM extracts structured constraints from the instruction (this is the only LLM step, and it can only push toward ESCALATE). Code then compares those keys against the catalog fields. If "width" was requested and the listing has no width field, the verdict is ESCALATE with a message naming the missing field. This stops your own Case #17 before payment instead of refunding it afterward, and it is easy to show on screen.

Order of operations. Run 1, 2, 3, 4 first because they need no network call. Then fetch the catalog record once and run 5, 6, 9, 10. Then 7, 8, 11. Log every check's result, pass or fail, not only the first failure, so the proposal detail screen can show all five red marks as in your Figure 9.

Defaults for the demo. Budget $400, per-transaction cap $400 in the first scenario, velocity limit 5 per hour, near-limit threshold 80%. These are starting values and should be tunable per session.

9. Recovery specification

Some mistakes pass every check. The human flags the purchase; the module gathers evidence from the audit log (instruction, agent reasoning, listing snapshot, order); an LLM picks a root cause; a rules table fixes the refund range; the model may choose a value inside the range and must explain it; the human accepts or rejects; on accept, code calls the refund endpoint.

Root cause	Range (your starting values)	Side effect
Seller fault (wrong item shipped)	90 to 100%	Seller pays return
Ambiguous listing (spec missing)	40 to 70%	Listing fix suggested
Agent error (clear instruction misread)	70 to 100%	Flag agent for review
Vague instruction	0 to 30%	Suggest clearer prompt

Making the ranges defensible. Fixed percentages look arbitrary to a judge. Borrow the economics from your Returnless idea: compute a suggested refund in code as `(avoided_return_freight + expected_resale_loss) / price`, then clamp it to the cause's range, and let the LLM only adjust within the clamp and write the explanation. Worked example, with assumed numbers: desk $349.00, avoided freight $60, resale loss 25% ($87.25). Suggested value is (60 + 87.25) / 349 = 42.2%, inside the 40 to 70% range for an ambiguous listing, so the proposal is 42% ($146.58). Label every economic input as an assumption in the UI.

Case #17 as drawn in your proposal. 60% of $349.00 is $209.40, correct arithmetic and inside 40 to 70%.

Safeguards. Refund amount at most capture minus prior refunds (PayPal also returns 422). Only one open case per payment. Every refund call carries its own `PayPal-Request-Id`. A rejected proposal packages the evidence for an optional formal dispute (Customer Disputes API); keep that off the critical path, and do not promise it works in sandbox (unverified).